Moving Average from Data Stream (easy)

Problem Statement

Given a stream of integers and window size `n`, calculate the moving average of all the integers of the sliding window.

Implement the `Solution` class:

- `MovingAverage(int size)` Initializes the object with the size of the window size, which is `n`.
- `double next(int x)` Calculates and returns the moving average of the last `n` values of the integer stream.

Examples

- **Example 1:**
  - **Input**: `["MovingAverage", "next", "next", "next"]`, `[[2], [1], [10], [3]]`
  - **Expected Output**: `[null, 1.0, 5.5, 6.5]`
  - **Justification**: The moving average of the last 2 numbers. Initially, it's 1. Then (1+10)/2 = 5.5, and (10+3)/2 = 6.5.
- **Example 2:**
  - **Input**: `["MovingAverage", "next", "next"]`, `[[4], [5], [15]]`
  - **Expected Output**: `[null, 5.0, 10.0]`
  - **Justification:** First, the average is 5. Then, (5+15)/2 = 10 as we consider the last 4 numbers, but only two numbers are in the stream.
- **Example 3:**
  - **Input**: `["MovingAverage", "next", "next", "next", "next"]`, `[[3], [7], [4], [8], [5]]`
  - **Expected Output**: `[null, 7.0, 5.5, 6.33, 5.67]`
  - **Justification**: Moving average calculations are (7), (7+4)/2, (7+4+8)/3, and (4+8+5)/3.

Try it yourself

Try solving this question here:

Java

```java
import java.util.LinkedList;
import java.util.Queue;
import java.util.List;
import java.util.Arrays;

public class Solution {
    // Constructor to initialize the moving window
    public Solution(int size) {
        // ToDo: Write Your Code Here.
    }

    // Method to add a new value and get the moving average
    public double next(int val) {
        // ToDo: Write Your Code Here.
        return 1.0;
    }
}

```

Test Case | Run Code Result | Stdout

```
["MovingAverage", "next", "next", "next", "next"]
[[3], [10], [20], [30], [40]]
```

Use Example Testcases | Run | Submit

← Previous
Solution: Island Perimeter

Next →
Solution: Moving Average From Data Stream

Mark as Completed